Experiment No. – 3				
Date of Performance:	**29/1/2026**			
Date of Submission:	**05/1/2026**			
Execution (06)	Viva (02)	Professional Ethics (02)	Experiment Total (10)	Sign with Date

Experiment No. 3

3.1 Aim: Install Metamask extension and web3 program

3.2 Course Outcome: Develop the application using the concept of Ethereum with smart contracts using different development tools.

3.3 Larning Objectives: Examine the working of Metamask and Web3

3.4 Requirement:

Hardware Requirements

- Computer/Laptop with minimum 4GB RAM
- Stable internet connection

Software Requirements

- Web browser (e.g., Google Chrome or Mozilla Firefox)
- MetaMask Extension
- Node.js (latest LTS version)
- Visual Studio Code
- Basic Web3 library (e.g., Web3.js or Ethers.js)

3.5 Related Theory:

Introduction to Web3

Web3 refers to the next generation of the internet where decentralized applications (DApps) interact with blockchain networks instead of centralized servers. It enables peer-to-peer transactions, smart contracts, and digital asset management without intermediaries.

Most Web3 applications are built on blockchain platforms like Ethereum, which supports smart contracts and decentralized applications.

MetaMask
MetaMask is a browser extension and crypto wallet that allows users to interact with blockchain networks directly from their browser.
It provides:

- Secure storage of private keys
- Account management
- Connection to decentralized applications (DApps)
- Transaction signing and verification

MetaMask acts as a bridge between the web browser and blockchain networks.

Web3 Development
Web3 development involves creating applications that interact with blockchain networks using JavaScript libraries.
Commonly used libraries include:

- Web3.js
- Ethers.js

These libraries allow developers to:

- Connect to blockchain nodes
- Read blockchain data
- Send transactions
- Interact with smart contracts

Node.js Environment
Node.js is required to install and manage Web3 libraries. It provides a runtime environment to execute JavaScript outside the browser and manage project dependencies using npm (Node Package Manager).

WEB3.JS:

Web3.js is a library that allows developers to interact with decentralized applications (dApps) built on the Ethereum blockchain using JavaScript. Some of the ways that web3.js can be used to create dApps include:Sending transactions: web3.js allows developers to create, sign, and send transactions on the Ethereum blockchain. Interacting with smart contracts: web3.js allows developers to interact with smart contracts on the Ethereum blockchain by calling their functions and reading their data. Retrieving blockchain data: web3.js allows developers to retrieve information from the Ethereum blockchain such as the current block number, the balance of an Ethereum address, and the state of a smart contract.

3.6 Procedure:

Creating and managing Ethereum accounts: web3.js allows developers to create and manage Ethereum accounts, including generating private keys, signing messages, and encrypting private keys.

Step 1: Go to Chrome Web Store Extensions Section.

Step 2: Search *MetaMask.*

Step 3: Check the number of downloads to make sure that the legitimate MetaMask is being installed, as hackers might try to make clones of it.

Step 4: Click the *Add to Chrome* button.

Step 5: Once installation is complete this page will be displayed. Click on the *Get Started* button.

Step 6: This is the first time creating a wallet, so click the *Create a Wallet* button. If there is a wallet then import the already created using the *Import Wallet* button.

Step 7: Click *I Agree* button to allow data to be collected to help improve MetaMask or else click the *No Thanks* button. The wallet can still be created even if the user will click on the *No Thanks* button.

Step 8: Create a password for your wallet. This password is to be entered every time the browser is launched and wants to use MetaMask. A new password needs to be created if chrome is uninstalled or if there is a switching of browsers. In that case, go through the *Import Wallet* button. This is because MetaMask stores the keys in the browser. Agree to *Terms of Use*.

Step 9: Click on the dark area which says *Click here to reveal secret words* to get your secret phrase.

Step 10: This is the most important step. Back up your secret phrase properly. Do not store your secret phrase on your computer. Please read everything on this screen until you understand it completely before proceeding. The secret phrase is the only way to access your wallet if you forget your password. Once done click the *Next* button.

Step 11: Click the buttons respective to the order of the words in your seed phrase. In other words, type the seed phrase using the button on the screen. If done correctly the *Confirm* button should turn blue.

Step 12: Click the *Confirm* button. Please follow the tips mentioned.

Step 13: One can see the balance and copy the address of the account by clicking on the *Account 1* area.

Step 14: One can access MetaMask in the browser by clicking the Foxface icon on the top right. If the Foxface icon is not visible, then click on the puzzle piece icon right next to it.

3.7 Program and Output:

STEP 1: OPEN REMIX IDE AND WRITE THE SMART CONTRACT

STEP 2: Select Ganache provider in remix environment. And enter RPC SERVER URL from Ganache

STEP 3: Deploy the Smart Contract.

STEP 4: Create a folder and then in that folder install Web3.
Use the command: **npm install web3**

STEP 5: Open the Terminal.
Open node terminal by using "node" command.
Define the Web3
let Web3 = require("web3");

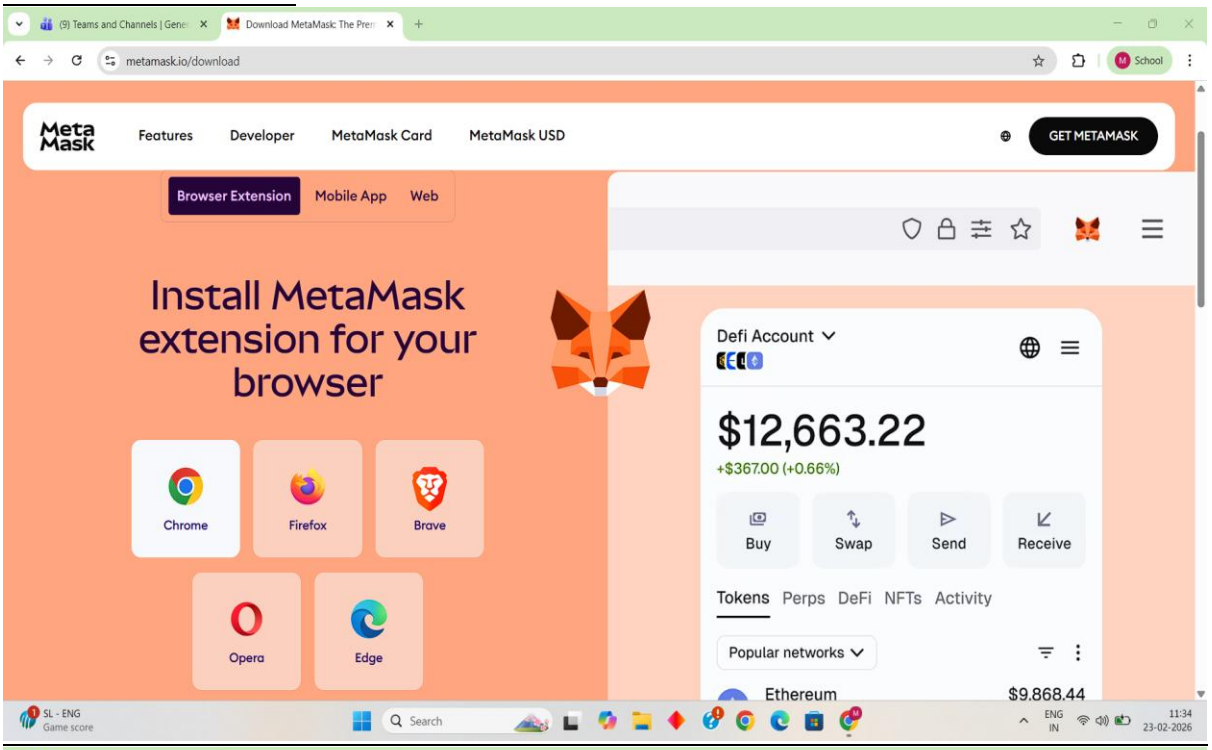
STEP 6: Connect to Ganache
let web3 = new Web3(new
Web3.providers.HttpProvider("HTTP://127.0.0.1:7545"));

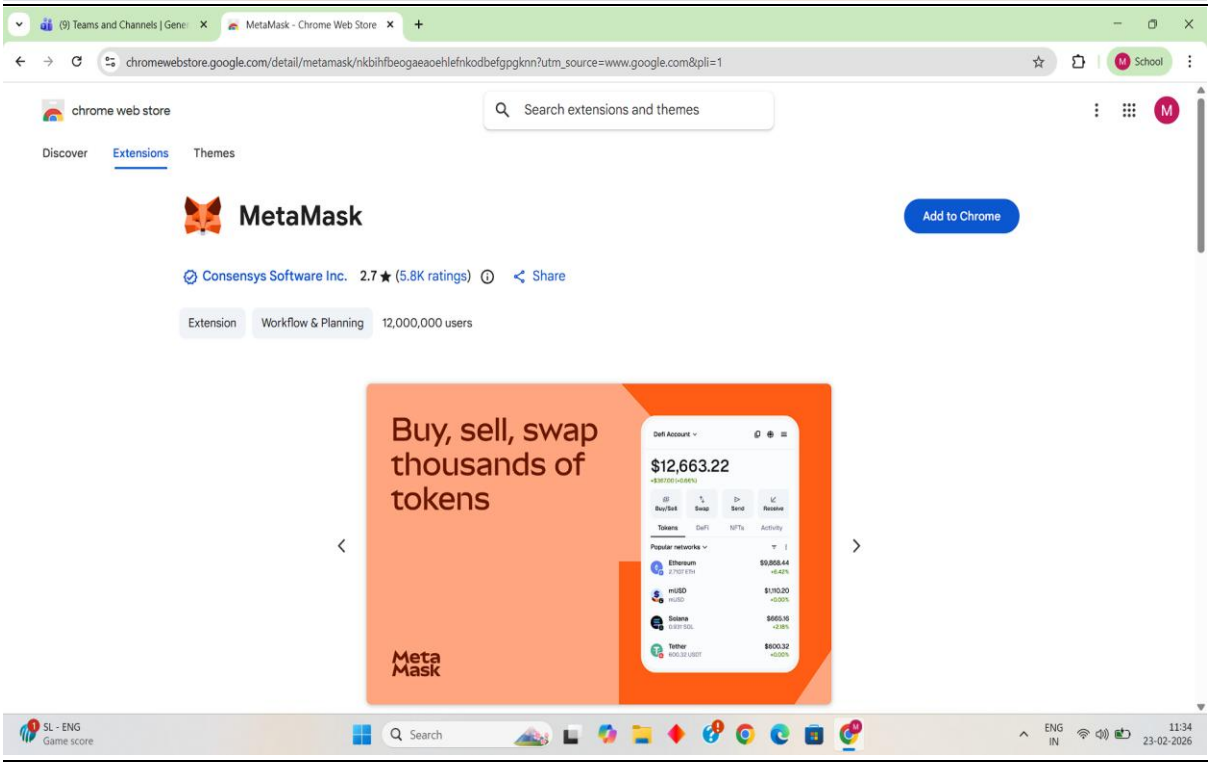
STEP 7: Enter ABI of contract and address. Instantiating a Web3.js Contract
let contracts = new web3.eth.Contract(ABI,"CONTRACT ADDRESS");
Copy ABI from compile section of remix
Copy CONTRACT ADDRESS from deploy section of remix and paste int in "" double quotes.

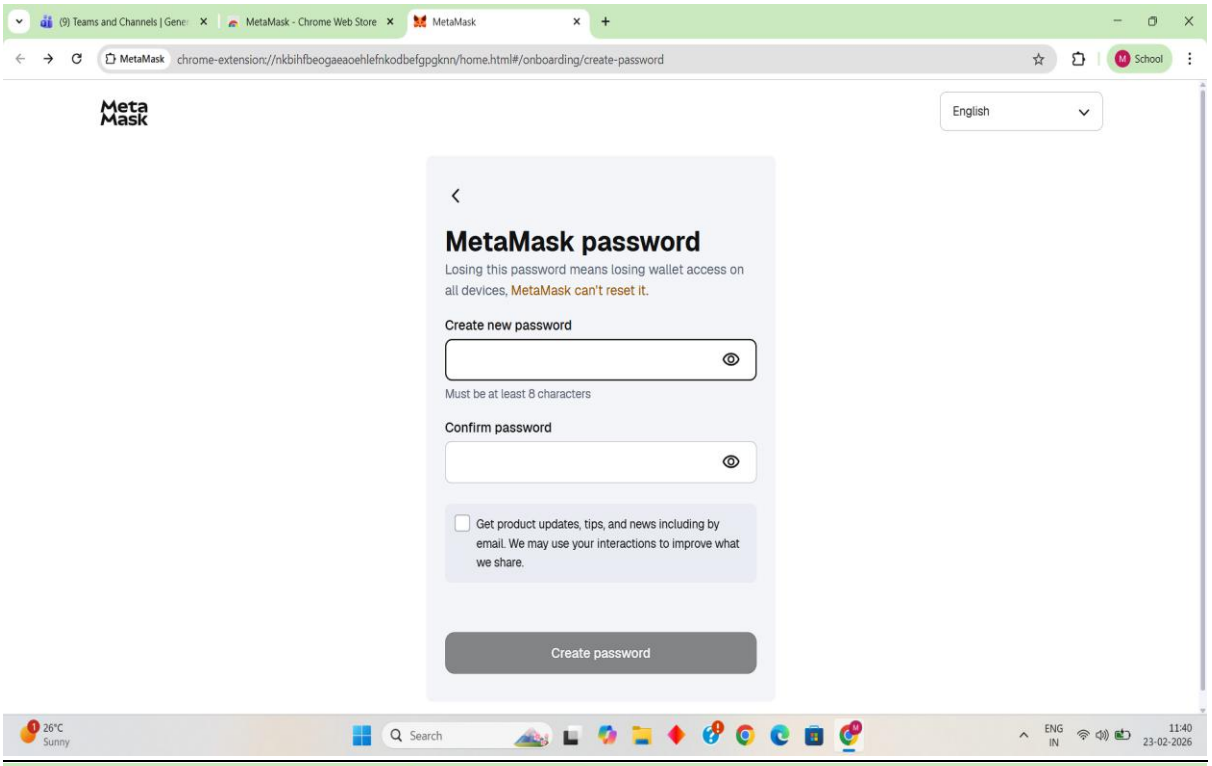
STEP 8: Call the function
contracts.methods.x().call().then(console.log);

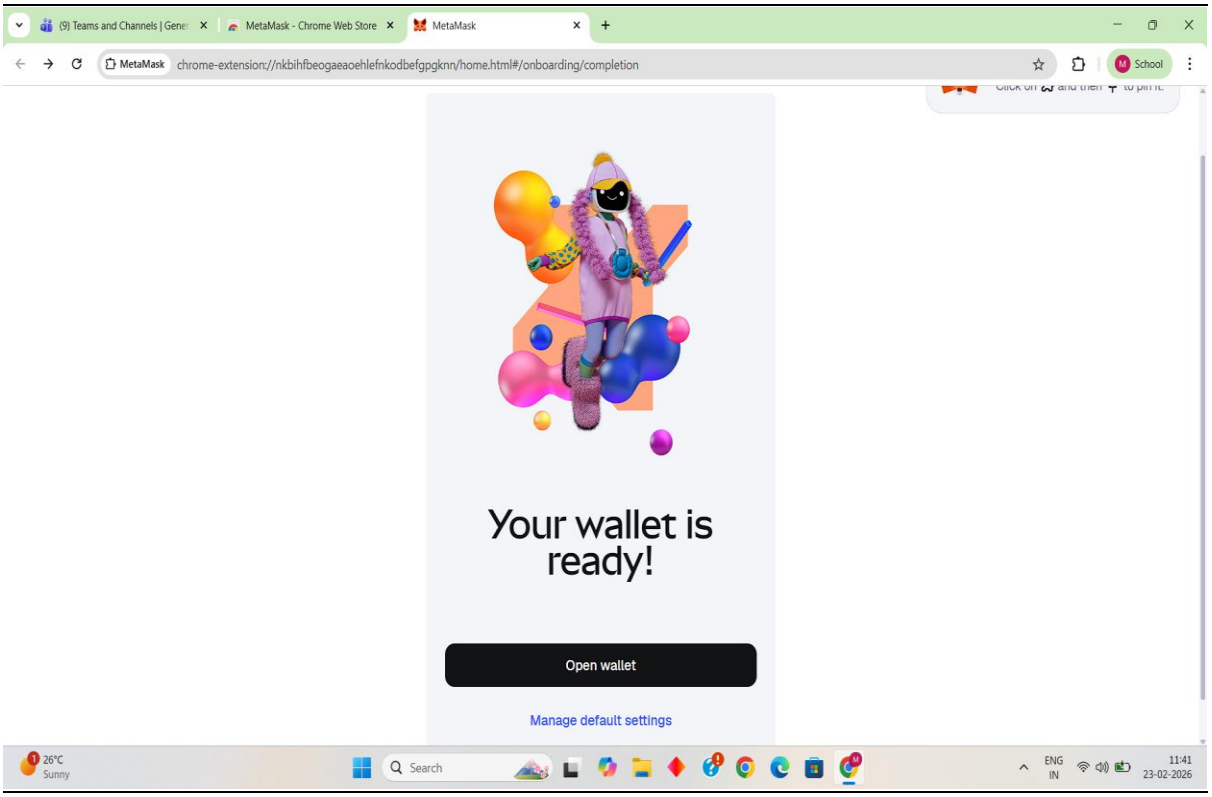
STEP 9: Enter value in the Function
contracts.methods.set(90).send({from:"enter ganache account"})

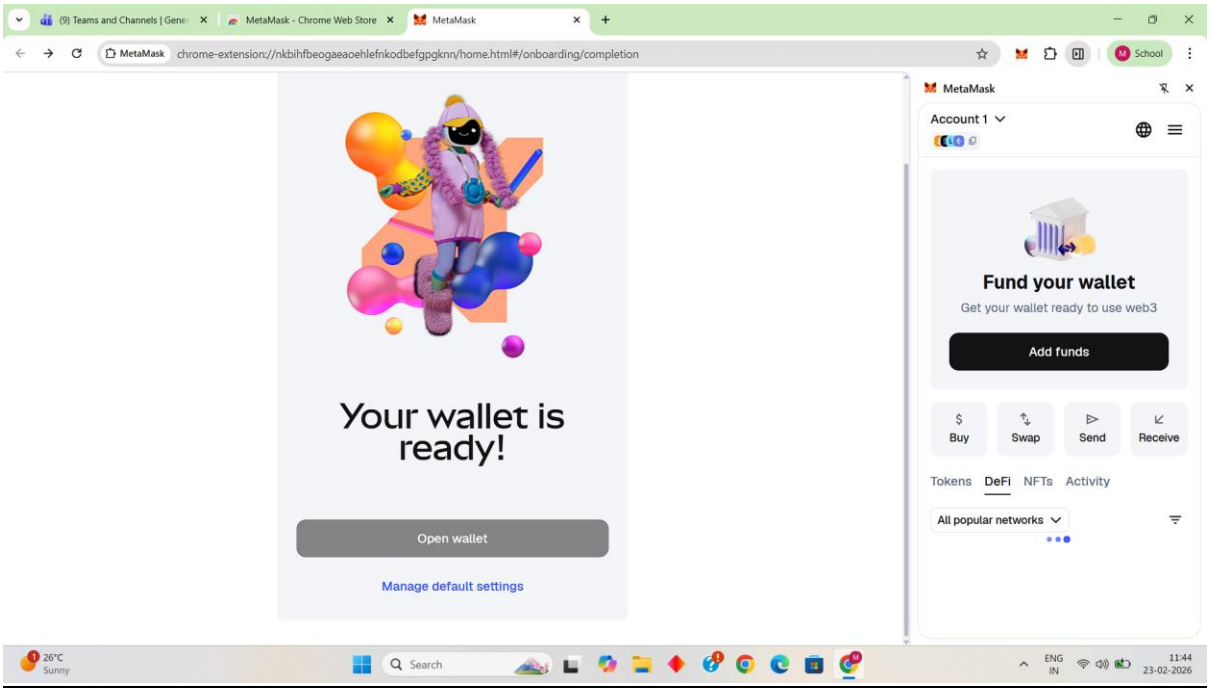
Metamask installation:



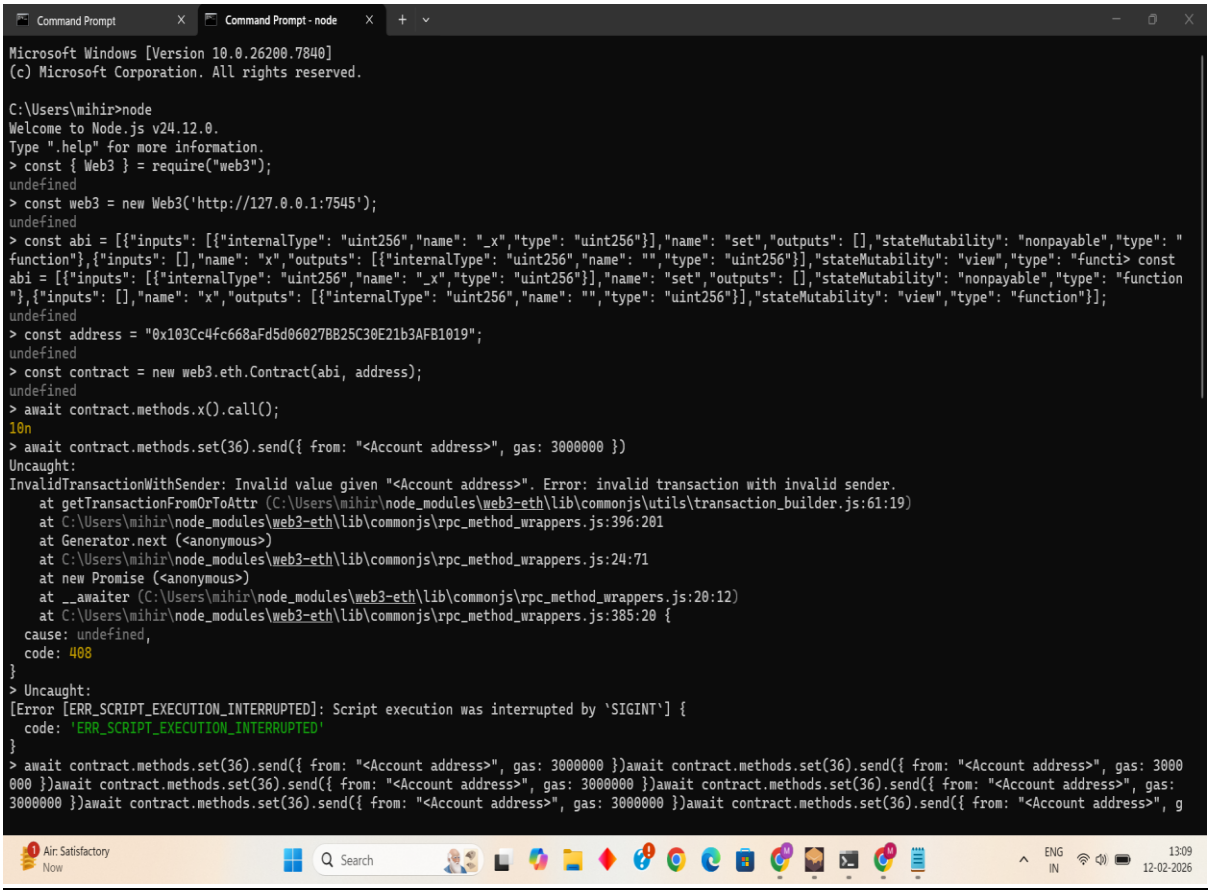



(9) Teams and Channels | Gene
MetaMask - Chrome Web Store
MetaMask
chrome-extension://nkbihfbeogaeaoehlefnkodbefgpgknn/home.html#/onboarding/create-password
School
Meta Mask
English
MetaMask password
Losing this password means losing wallet access on all devices, MetaMask can't reset it.
Create new password
Must be at least 8 characters
Confirm password
Get product updates, tips, and news including by email. We may use your interactions to improve what we share.
Create password
26°C Sunny
Search
ENG IN
11:40 23-02-2026


(9) Teams and Channels | Gene
MetaMask - Chrome Web Store
MetaMask
chrome-extension://nkbihfbeogaeaoehlefnkodbefgpgknn/home.html#/onboarding/completion
School
Your wallet is ready!
Open wallet
Manage default settings
26°C Sunny
Search
ENG IN
11:41 23-02-2026

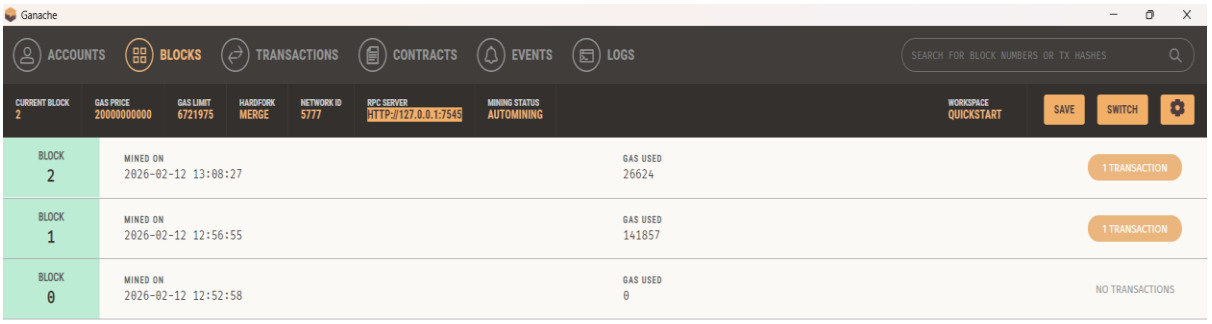


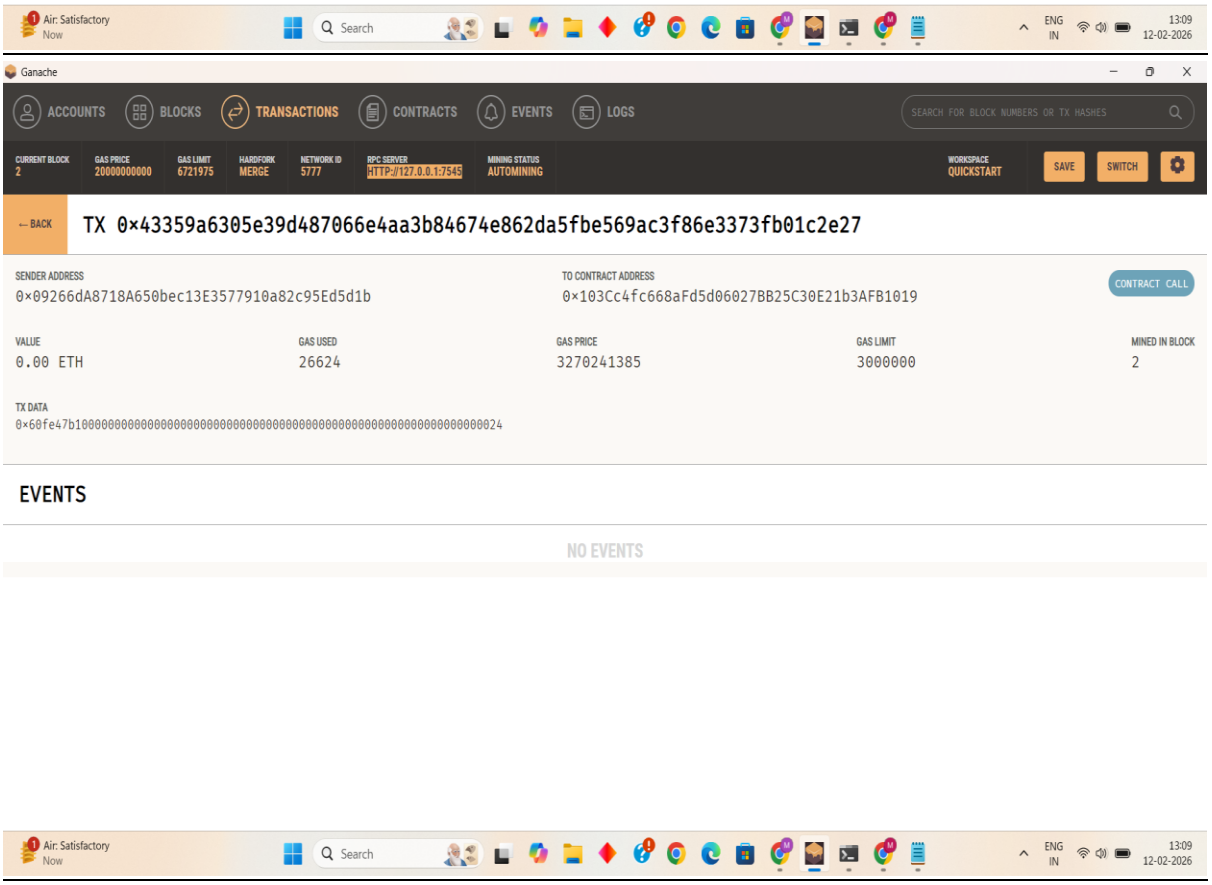
web3 program:

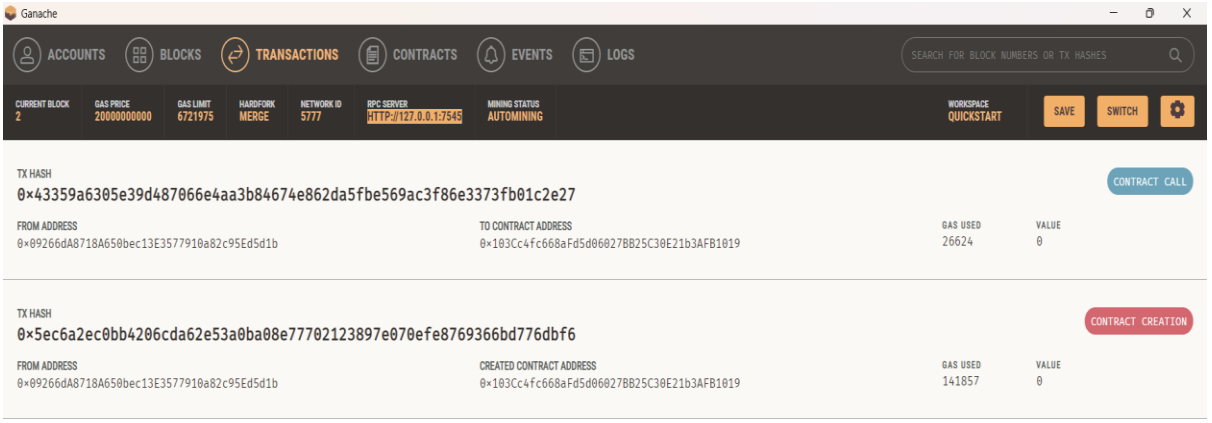


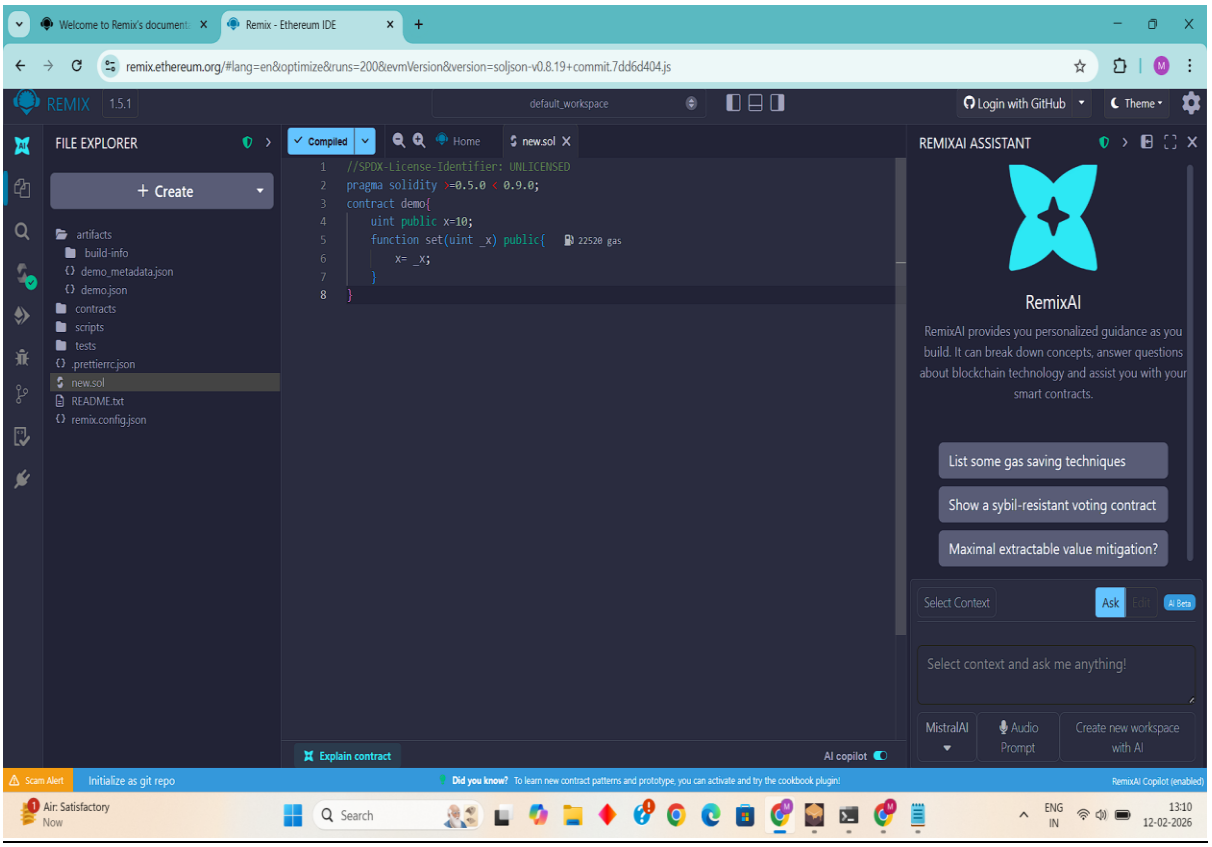
```
Microsoft Windows [Version 10.0.26200.7840]
(c) Microsoft Corporation. All rights reserved.

C:\Users\mihir>node
Welcome to Node.js v24.12.0.
Type ".help" for more information.
> const { Web3 } = require("web3");
undefined
> const web3 = new Web3('http://127.0.0.1:7545');
undefined
> const abi = [{"inputs": [{"internalType": "uint256","name": "_x","type": "uint256"}],"name": "set","outputs": [],"stateMutability": "nonpayable","type": "
function"},{"inputs": [],"name": "x","outputs": [{"internalType": "uint256","name": "","type": "uint256"}],"stateMutability": "view","type": "functi> const
abi = [{"inputs": [{"internalType": "uint256","name": "_x","type": "uint256"}],"name": "set","outputs": [],"stateMutability": "nonpayable","type": "function
"},{"inputs": [],"name": "x","outputs": [{"internalType": "uint256","name": "","type": "uint256"}],"stateMutability": "view","type": "function"}];
undefined
> const address = "0x103Cc4fc668aFd5d06027BB25C30E21b3AFB1019";
undefined
> const contract = new web3.eth.Contract(abi, address);
undefined
> await contract.methods.x().call();
10n
> await contract.methods.set(36).send({ from: "<Account address>", gas: 3000000 })
Uncaught:
InvalidTransactionWithSender: Invalid value given "<Account address>". Error: invalid transaction with invalid sender.
    at getTransactionFromOrToAttr (C:\Users\mihir\node_modules\web3-eth\lib\commonjs\utils\transaction_builder.js:61:19)
    at C:\Users\mihir\node_modules\web3-eth\lib\commonjs\rpc_method_wrappers.js:396:201
    at Generator.next (<anonymous>)
    at C:\Users\mihir\node_modules\web3-eth\lib\commonjs\rpc_method_wrappers.js:24:71
    at new Promise (<anonymous>)
    at __awaiter (C:\Users\mihir\node_modules\web3-eth\lib\commonjs\rpc_method_wrappers.js:20:12)
    at C:\Users\mihir\node_modules\web3-eth\lib\commonjs\rpc_method_wrappers.js:385:20 {
  cause: undefined,
  code: 408
}
> Uncaught:
[Error [ERR_SCRIPT_EXECUTION_INTERRUPTED]: Script execution was interrupted by `SIGINT`] {
  code: 'ERR_SCRIPT_EXECUTION_INTERRUPTED'
}
> await contract.methods.set(36).send({ from: "<Account address>", gas: 3000000 })await contract.methods.set(36).send({ from: "<Account address>", gas: 3000
000 })await contract.methods.set(36).send({ from: "<Account address>", gas: 3000000 })await contract.methods.set(36).send({ from: "<Account address>", gas:
3000000 })await contract.methods.set(36).send({ from: "<Account address>", gas: 3000000 })await contract.methods.set(36).send({ from: "<Account address>", g
```

```
Command Prompt | Command Prompt - node

hods.set(36).send({ from: "<Account address>", gas: 3000000 })await contract.methods.set(36).send({ from: "<Account address>", gas: 3000000 })await contract
.methods.set(36).send({ from: "<Account address>", gas: 3000000 })await contract.methods.set(36).send({ from: "<Account address>", gas: 3000000 })await cont
ract.methods.set(36).send({ from: "<Account address>", gas: 3000000 })await contract.methods.set(36).send({ from: "<Account address>", gas: 3000000 })await
contract.methods.set(36).send({ from: "<Account address>", gas: 3000000 })await contract.methods.set(36).send({ from: "<Account address>", gas: 3000000 })aw
ait contract.methods.set(36).send({ from: "<Account address>", gas: 3000000 })await contract.methods.set(36).send({ from: "<Account address>", gas: 3000000
})await contract.methods.set(36).send({ from: "<Account address>", gas: 3000000 })aawawawaw
> await contract.methods.set(36).send({ from: "<Account address>", gas: 3000> await contract.methods.set(36).send({ from: "0x09266dA8718A650bec13E357791> aw
ait contract.methods.set(36).send({ from: "0x09266dA8718A650bec13E3577910a82c95Ed5d1b", gas: 3000000 })
{
  transactionHash: '0x43359a6305e39d487066e4aa3b84674e862da5fbe569ac3f86e3373fb01c2e27',
  transactionIndex: 0n,
  blockNumber: 2n,
  blockHash: '0x247e9e077262098e15653f91de32953e77bab0784466485127694edecfb5b02c',
  from: '0x09266da8718a650bec13e3577910a82c95ed5d1b',
  to: '0x103cc4fc668afd5d06027bb25c30e21b3afb1019',
  cumulativeGasUsed: 26624n,
  gasUsed: 26624n,
  logs: [],
  logsBloom: '0x0000000000
```


Ganache
ACCOUNTS
BLOCKS
TRANSACTIONS
CONTRACTS
EVENTS
LOGS
SEARCH FOR BLOCK NUMBERS OR TX HASHES
CURRENT BLOCK 2
GAS PRICE 20000000000
GAS LIMIT 6721975
HARDFORK MERGE
NETWORK ID 5777
RPC SERVER HTTP://127.0.0.1:7545
MINING STATUS AUTOMINING
WORKSPACE QUICKSTART
SAVE
SWITCH
BLOCK 2
MINED ON 2026-02-12 13:08:27
GAS USED 26624
1 TRANSACTION
BLOCK 1
MINED ON 2026-02-12 12:56:55
GAS USED 141857
1 TRANSACTION
BLOCK 0
MINED ON 2026-02-12 12:52:58
GAS USED 0
NO TRANSACTIONS


Air: Satisfactory
Now
Search
ENG IN
13:09 12-02-2026
Ganache
ACCOUNTS
BLOCKS
TRANSACTIONS
CONTRACTS
EVENTS
LOGS
SEARCH FOR BLOCK NUMBERS OR TX HASHES
CURRENT BLOCK 2
GAS PRICE 20000000000
GAS LIMIT 6721975
HARDFORK MERGE
NETWORK ID 5777
RPC SERVER HTTP://127.0.0.1:7545
MINING STATUS AUTOMINING
WORKSPACE QUICKSTART
SAVE
SWITCH
← BACK
TX 0x43359a6305e39d487066e4aa3b84674e862da5fbe569ac3f86e3373fb01c2e27
SENDER ADDRESS
0x09266dA8718A650bec13E3577910a82c95Ed5d1b
TO CONTRACT ADDRESS
0x103Cc4fc668aFd5d06027BB25C30E21b3AFB1019
CONTRACT CALL
VALUE
0.00 ETH
GAS USED
26624
GAS PRICE
3270241385
GAS LIMIT
3000000
MINED IN BLOCK
2
TX DATA
0x60fe47b1000000000000000000000000000000000000000000000000000000000000000024
EVENTS
NO EVENTS
Air: Satisfactory
Now
Search
ENG IN
13:09 12-02-2026





3.8 Conclusion:

This experiment, introduced the MetaMask browser extension, a popular non-custodial wallet for interacting with Ethereum-based decentralized applications (dApps). Participants installed the extension, set up a secure test account, and performed a Web3 program, likely involving connecting to a test network (e.g., Goerli or Sepolia), checking balances, or interacting with a simple smart contract.